

RecoMart Retailrocket Recommendation Pipeline

Data Management for Machine Learning - Assignment I
End-to-End Data Management Pipeline for a Recommendation System

This PDF is the single consolidated documentation report. It covers problem formulation, data ingestion, raw storage, validation, preparation, EDA, SQL transformation, feature store, DVC versioning, MLflow tracking, model training, inference, orchestration, and reproducibility.

1. Assignment Requirement Coverage

Requirement	Status	Evidence
Problem formulation	Covered	Business problem, objectives, outputs, metrics
Data collection and ingestion	Covered	Retailrocket batch CSV data and REST/mock catalog metadata delta source
Raw data storage	Covered	Partitioned data lake using source/type/ingestion_timestamp
Data profiling and validation	Covered	Raw, staged, and curated validation scripts and reports
Data preparation and EDA	Covered	Staged Parquet, curated datasets, and summary plots
Feature engineering and transformation	Covered	DuckDB SQL warehouse and feature tables
Feature store	Covered	Custom registry and retrieval demo
Data versioning and lineage	Covered	DVC metadata and versioning workflow
Model training and evaluation	Covered	Retailrocket popularity baseline and content-based recommender with HitRate, ...
Pipeline orchestration	Covered	Prefect flows and successful run logs
Documentation and demo	Covered	Single PDF report, screenshots, video script

2. Problem Formulation

RecoMart is an e-commerce startup that wants product recommendations to improve customer engagement, conversion, and cross-selling. The pipeline produces clean datasets, engineered features, trained recommendation models, inference outputs, and reproducible metadata for monitoring and lineage.

- Business problem: recommend relevant products to users based on interaction history and item metadata.
- Expected outputs: clean EDA-ready datasets, feature tables, trained model artifact, inference interface.
- Evaluation metrics: Precision@K, Recall@K, HitRate@K, and NDCG@K.
- Implemented models: Retailrocket event-weighted popularity baseline and category/content-based recommender.

3. Data Sources

3.1 Retailrocket External Dataset

Retailrocket is the main large-scale dataset. It includes visitor-item events, item properties, and category hierarchy data.

dataset_name	file_path	file_size_mb	row_count	distinct_visitors	distinct_items	distinct_properties	distinct_categories
events	data/external/retailrocket/events.csv	89.872	2756101.0	1407580.0	235061.0		
item_properties_part1	data/external/retailrocket/item_properties_part1.csv	461.879	10999999.0		417053.0	1097.0	
category_tree	data/external/retailrocket/category_tree.csv	0.014	1669.0				1669.0

event	row_count	distinct_visitors	distinct_items	rows_with_transaction_id
view	2664312	1404179	234838	0.0
addtocart	69332	37722	23903	0.0
transaction	22457	11719	12025	22457.0

4. Ingestion, Raw Storage, and Logging

The ingestion layer contains Retailrocket batch ingestion and Retailrocket REST/mock catalog metadata deltas. Retailrocket source data is tracked by DVC as an external dataset, and raw ingestion snapshots are stored in a structured local data lake layout. The teammate-hosted real API returns non-repeated rows using the data/success/count/total_rows/unread_rows contract, while mock mode remains the reproducible local fallback using records/next_cursor/has_more.

- Retailrocket inspection script: src/ingestion/inspect_retailrocket_external.py
- Retailrocket batch ingestion script: src/ingestion/ingest_retailrocket_batch.py
- Retailrocket API ingestion script: src/ingestion/ingest_retailrocket_catalog_api.py
- Mock API server: src/ingestion/mock_retailrocket_catalog_api.py
- Default API source: https://recomart-flask.295uyonmxxer.us-south.codeengine.appdomain.cloud/items.
- API page size: RECOMART_CATALOG_API_PAGE_SIZE defaults to 50 and is sent as the count query parameter.
- Raw layout: data/raw/source=<source_name>/type=<data_type>/ingestion_timestamp=<YYYYMMDD_HHMMSS>/
- Raw batch source: data/raw/source=retailrocket_batch/
- Raw API source: data/raw/source=retailrocket_api/
- External storage layout: data/external/retailrocket

5. Data Validation and Quality Reports

5.1 Raw Validation

status	check_count
PASS	99

5.2 Staged and Curated Validation

status	check_count
PASS	59
WARN	3

Validation checks include missing values, duplicate IDs, schema mismatch, valid event types, transaction ID consistency, latest item metadata uniqueness, and metadata coverage. The current reports contain no failing validation checks.

Warning-level checks are documented and do not block the pipeline. Current warnings are related to duplicate event groups and item metadata coverage for category and availability fields.

dataset_name	check_name	status	details
events	duplicate_event_rows	WARN	Duplicate event groups: 458
referential_coverage	event_items_with_category	WARN	102390/235061 (0.4356)
referential_coverage	event_items_with_availability	WARN	108603/235061 (0.4620)

6. Data Preparation and EDA Plots

Raw CSV/API data is converted into clean staged Parquet datasets and curated analytical datasets. EDA artifacts summarize event distribution, item popularity, user activity distribution, and model metrics.

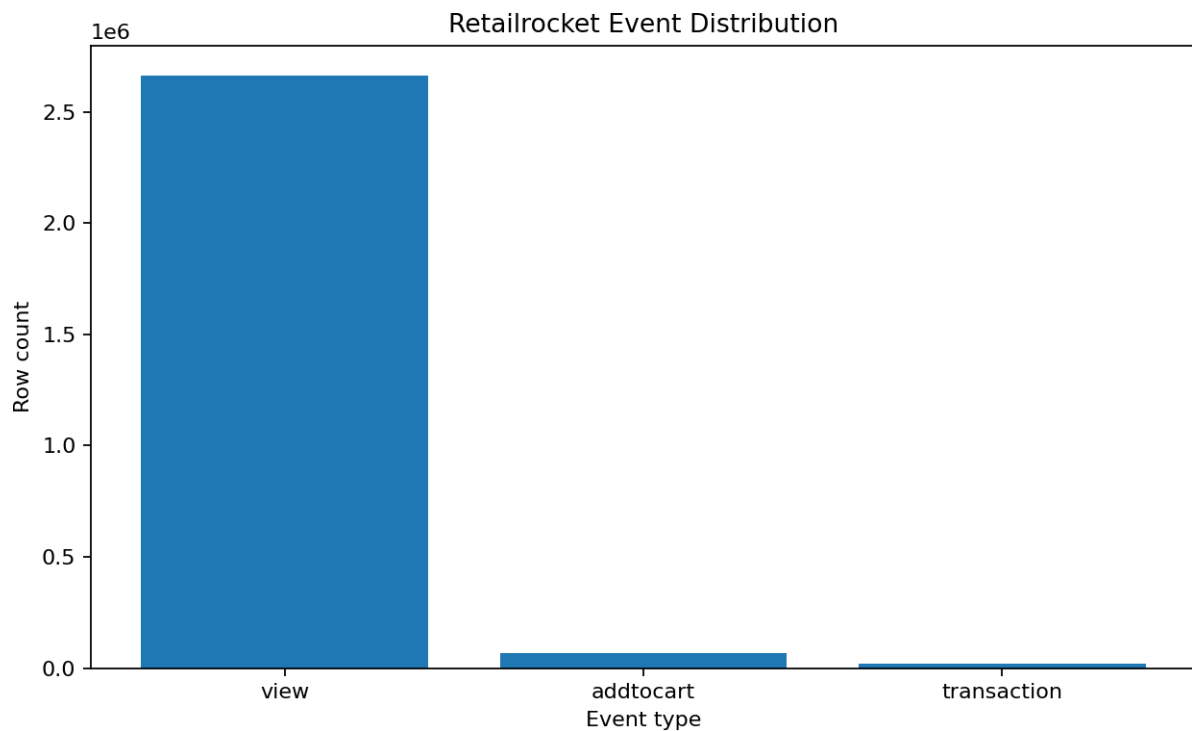
6.1 Staged Preparation Summary

dataset_name	output_file	description	source_files	row_count	column_count	file_size_mb
events	data/staged/source=retailrocket/events.parquet	Retailrocket visitor-item events prepared from raw batch ingestion.	data/raw/source=retailrocket_batch/type=events/ingestion_timestamp=20260428_2...	2756101	8	54.964
category_tree	data/staged/source=retailrocket/category_tree.parquet	Retailrocket category hierarchy prepared from raw batch ingestion.	data/raw/source=retailrocket_batch/type=category_tree/ingestion_timestamp=202...	1669	5	0.012
item_properties_batch	data/staged/source=retailrocket/item_properties_batch.parquet	Selected Retailrocket batch item properties from raw item_properties_part1.	data/raw/source=retailrocket_batch/type=item_properties_part1/ingestion_times...	1243692	8	8.857

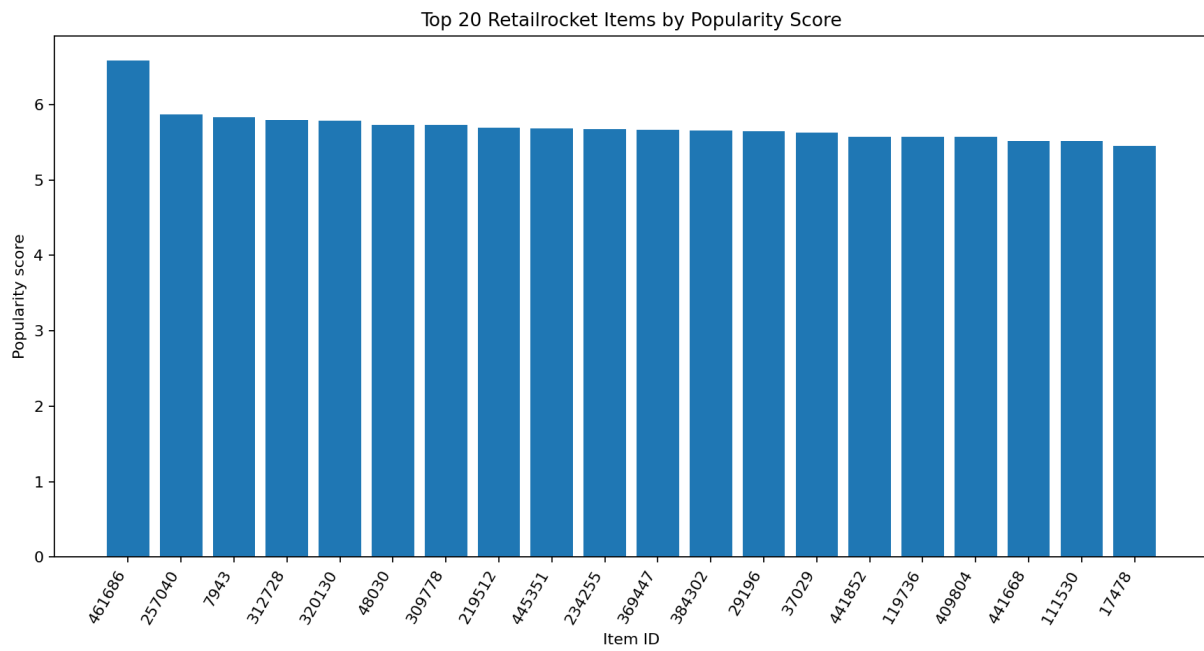
dataset_name	output_file	description	source_files	row_count	column_count	file_size_mb
item_properties_api_delta	data/staged/source=retailrocket/item_properties_api_delta.parquet	Retailrocket item property delta records from all REST/mock API raw landings.	data/raw/source=retailrocket_api/type=item_properties_delta/ingestion_timesta...	260	9	0.007
item_properties_combined	data/staged/source=retailrocket/item_properties_combined.parquet	Combined batch and API delta item properties.		1243952	9	8.864
item_category_latest	data/staged/source=retailrocket/item_category_latest.parquet	Latest categoryid value per item from combined item properties.		229346	6	1.499
item_availability_latest	data/staged/source=retailrocket/item_availability_latest.parquet	Latest availability value per item from combined item properties.		236885	7	1.295

6.2 Curated Dataset Summary

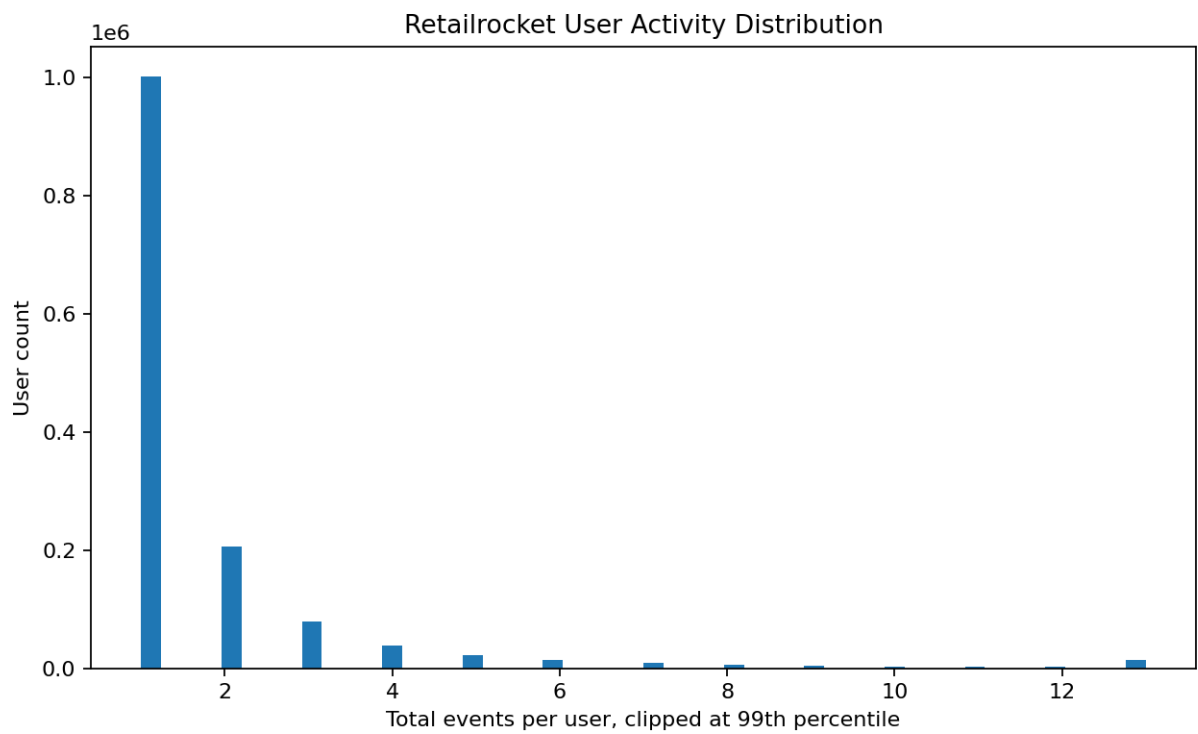
dataset_name	output_file	description	source_files	row_count	column_count	file_size_mb
curated_interactions	data/curated/source=retailrocket/curated_interactions.parquet	EDA-ready Retailrocket interaction rows enriched with latest item metadata.		2756101	13	60.749
curated_items	data/curated/source=retailrocket/curated_items.parquet	EDA-ready Retailrocket item dataset with event aggregates and metadata.		235061	16	8.527
curated_user_item_interactions	data/curated/source=retailrocket/curated_user_item_interactions.parquet	EDA/modeling-ready user-item interaction aggregates.		2145179	19	80.793



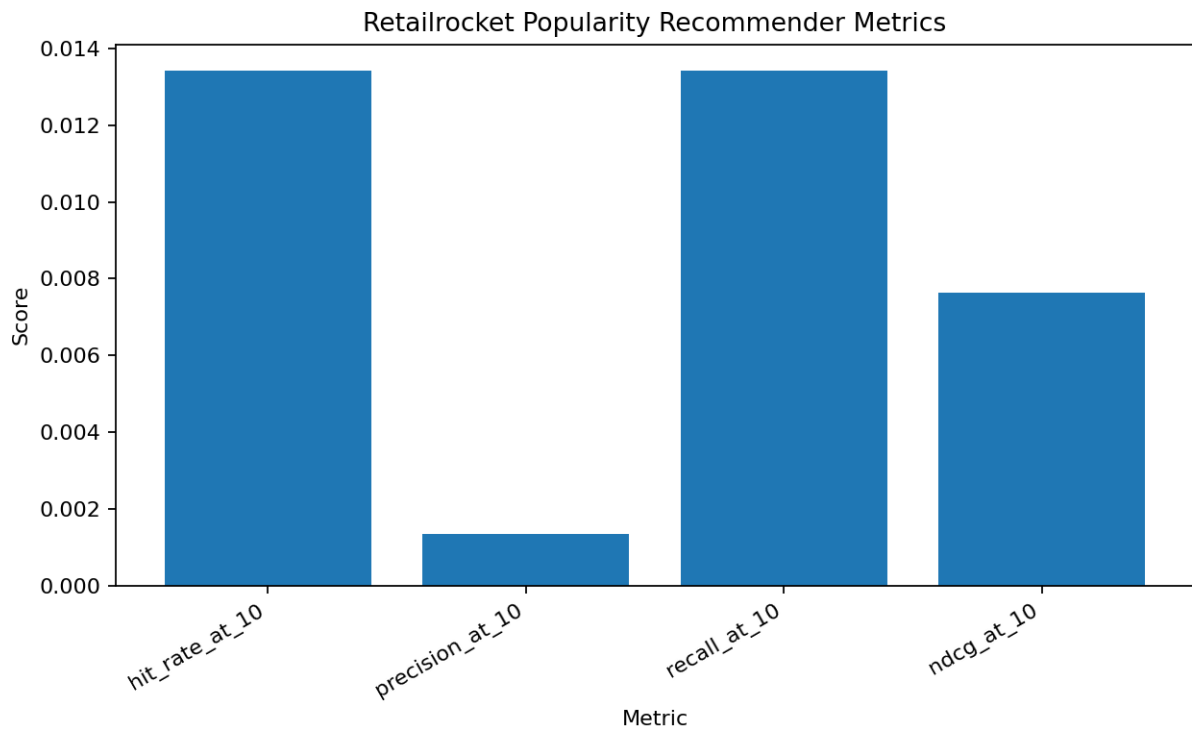
Retailrocket event distribution.



Top Retailrocket items by popularity score.



Retailrocket user activity distribution.



Retailrocket model metrics.

7. SQL Warehouse Schema and Transformation

DuckDB is used as a local analytical warehouse. Staged Parquet files are loaded into staged tables and transformed into mart and feature tables.

schema_name	relation_name	relation_type	row_count	column_count	status
staged	retailrocket_events	BASE TABLE	2756101	8	PASS
staged	retailrocket_category_tree	BASE TABLE	1669	5	PASS
staged	retailrocket_item_properties_batch	BASE TABLE	1243692	8	PASS
staged	retailrocket_item_properties_api_delta	BASE TABLE	260	9	PASS
staged	retailrocket_item_properties_combined	BASE TABLE	1243952	9	PASS
staged	retailrocket_item_category_latest	BASE TABLE	229346	6	PASS
staged	retailrocket_item_availability_latest	BASE TABLE	236885	7	PASS
curated	retailrocket_interactions	BASE TABLE	2756101	13	PASS
curated	retailrocket_items	BASE TABLE	235061	16	PASS
curated	retailrocket_user_item_interactions	BASE TABLE	2145179	19	PASS
mart	retailrocket_interactions	VIEW	2756101	13	PASS
mart	retailrocket_user_item_interactions	BASE TABLE	2145179	19	PASS

7.1 SQL Schema Sample

schema_name	table_name	column_name	ordinal_position	data_type	is_nullable
curated	retailrocket_interactions	user_id	1	BIGINT	YES
curated	retailrocket_interactions	item_id	2	BIGINT	YES

schema_name	table_name	column_name	ordinal_position	data_type	is_nullable
curated	retailrocket_interactions	event_type	3	VARCHAR	YES
curated	retailrocket_interactions	event_timestamp_ms	4	BIGINT	YES
curated	retailrocket_interactions	event_datetime	5	TIMESTAMP WITH TIME ZONE	YES
curated	retailrocket_interactions	transaction_id	6	BIGINT	YES
curated	retailrocket_interactions	interaction_weight	7	DECIMAL(2,1)	YES
curated	retailrocket_interactions	category_id	8	BIGINT	YES
curated	retailrocket_interactions	parent_category_id	9	BIGINT	YES
curated	retailrocket_interactions	is_available	10	INTEGER	YES
curated	retailrocket_interactions	available_raw_value	11	VARCHAR	YES
curated	retailrocket_interactions	has_category_metadata	12	INTEGER	YES
curated	retailrocket_interactions	has_availability_metadata	13	INTEGER	YES
curated	retailrocket_items	item_id	1	BIGINT	YES
curated	retailrocket_items	total_events	2	BIGINT	YES
curated	retailrocket_items	unique_users	3	BIGINT	YES
curated	retailrocket_items	view_events	4	DOUBLE	YES
curated	retailrocket_items	addtocart_events	5	DOUBLE	YES

8. Feature Engineering Logic

Feature engineering creates user-level, item-level, and user-item interaction features for recommendation.

feature_group	feature_or_group	source	logic	used_for
retailrocket_user_features	total_events	mart.retailrocket_interactions	Count all events generated by each user.	User activity representation, training, inference analysis
retailrocket_user_features	unique_items	mart.retailrocket_interactions	Count distinct items interacted with by each user.	User diversity and activity profiling
retailrocket_user_features	view_events, addtocart_events, transaction_events	mart.retailrocket_interactions	Conditional event counts by user and event type.	Intent-strength measurement
retailrocket_user_features	addtocart_event_rate, transaction_event_rate	features.retailrocket_user_features	Event-type count divided by total user event count.	User conversion tendency
retailrocket_item_features	total_events, unique_users	mart.retailrocket_interactions	Count total item interactions and distinct users per item.	Item popularity and candidate ranking
retailrocket_item_features	category_id, parent_category_id	staged.retailrocket_item_category_latest, staged.retailrocket_category_tree	Attach latest item category and parent category metadata.	Content-aware filtering and catalog analysis
retailrocket_item_features	is_available	staged.retailrocket_item_availability_latest	Attach latest availability flag for each item.	Candidate filtering and item metadata enrichment
retailrocket_item_features	addtocart_rate, transaction_rate, cart_to_transaction_rate	features.retailrocket_item_features	Conversion ratios derived from item-level event counts.	Ranking and item quality signals
retailrocket_user_item_features	view_count, addtocart_count, transaction_count	mart.retailrocket_user_item_interactions	Aggregate user-item event counts by event type.	Implicit feedback model training
retailrocket_user_item_features	implicit_label	features.retailrocket_user_item_features	Encode interaction strength: view < addtocart < transaction.	Recommendation target signal
retailrocket_user_item_features	strongest_event_type	features.retailrocket_user_item_features	Select strongest observed user-item event type.	Interpretability and behavioral analysis
retailrocket_feature_registry	user, item, user-item feature views	configs/feature_store/retailrocket_feature_registry.json	Custom registry maps Retailrocket feature views to entities, source paths, ve...	Feature store demonstration

9. Feature Store and Feature Metadata

A custom feature registry is implemented to demonstrate feature store concepts. Feature metadata documents feature views, entities, source paths, column roles, data types, and version.

9.1 Sample Feature Retrieval Demonstration

retrieval_type	feature_view_name	feature_view_version	entity_filter	rows_returned
user_features	retailrocket_user_features	v1	user_id=1369328	1
item_features	retailrocket_item_features	v1	item_id=356339	1
user_item_features	retailrocket_user_item_features	v1	user_id=1369328,item_id=356339	1

9.2 Feature Metadata Documentation Sample

registry_name	registry_version	feature_view_name	feature_view_version	entity_keys	source_path	source_table
retailrocket_recommendation_feature_registry	v1	retailrocket_user_features	v1	["user_id"]	data/features/source=retailrocket/user_features.parquet	features.retailrocket_user_features
retailrocket_recommendation_feature_registry	v1	retailrocket_user_features	v1	["user_id"]	data/features/source=retailrocket/user_features.parquet	features.retailrocket_user_features
retailrocket_recommendation_feature_registry	v1	retailrocket_user_features	v1	["user_id"]	data/features/source=retailrocket/user_features.parquet	features.retailrocket_user_features
retailrocket_recommendation_feature_registry	v1	retailrocket_user_features	v1	["user_id"]	data/features/source=retailrocket/user_features.parquet	features.retailrocket_user_features
retailrocket_recommendation_feature_registry	v1	retailrocket_user_features	v1	["user_id"]	data/features/source=retailrocket/user_features.parquet	features.retailrocket_user_features
retailrocket_recommendation_feature_registry	v1	retailrocket_user_features	v1	["user_id"]	data/features/source=retailrocket/user_features.parquet	features.retailrocket_user_features
retailrocket_recommendation_feature_registry	v1	retailrocket_user_features	v1	["user_id"]	data/features/source=retailrocket/user_features.parquet	features.retailrocket_user_features
retailrocket_recommendation_feature_registry	v1	retailrocket_user_features	v1	["user_id"]	data/features/source=retailrocket/user_features.parquet	features.retailrocket_user_features
retailrocket_recommendation_feature_registry	v1	retailrocket_user_features	v1	["user_id"]	data/features/source=retailrocket/user_features.parquet	features.retailrocket_user_features
retailrocket_recommendation_feature_registry	v1	retailrocket_user_features	v1	["user_id"]	data/features/source=retailrocket/user_features.parquet	features.retailrocket_user_features
retailrocket_recommendation_feature_registry	v1	retailrocket_user_features	v1	["user_id"]	data/features/source=retailrocket/user_features.parquet	features.retailrocket_user_features
retailrocket_recommendation_feature_registry	v1	retailrocket_user_features	v1	["user_id"]	data/features/source=retailrocket/user_features.parquet	features.retailrocket_user_features
retailrocket_recommendation_feature_registry	v1	retailrocket_user_features	v1	["user_id"]	data/features/source=retailrocket/user_features.parquet	features.retailrocket_user_features
retailrocket_recommendation_feature_registry	v1	retailrocket_user_features	v1	["user_id"]	data/features/source=retailrocket/user_features.parquet	features.retailrocket_user_features
retailrocket_recommendation_feature_registry	v1	retailrocket_user_features	v1	["user_id"]	data/features/source=retailrocket/user_features.parquet	features.retailrocket_user_features
retailrocket_recommendation_feature_registry	v1	retailrocket_user_features	v1	["user_id"]	data/features/source=retailrocket/user_features.parquet	features.retailrocket_user_features
retailrocket_recommendation_feature_registry	v1	retailrocket_user_features	v1	["user_id"]	data/features/source=retailrocket/user_features.parquet	features.retailrocket_user_features
retailrocket_recommendation_feature_registry	v1	retailrocket_user_features	v1	["user_id"]	data/features/source=retailrocket/user_features.parquet	features.retailrocket_user_features

registry_name	registry_version	feature_view_name	feature_view_version	entity_keys	source_path	source_table
retailrocket_recommendation_feature_registry	v1	retailrocket_item_features	v1	["item_id"]	data/features/source=retailrocket/item_features.parquet	features.retailrocket_item_features
retailrocket_recommendation_feature_registry	v1	retailrocket_item_features	v1	["item_id"]	data/features/source=retailrocket/item_features.parquet	features.retailrocket_item_features
retailrocket_recommendation_feature_registry	v1	retailrocket_item_features	v1	["item_id"]	data/features/source=retailrocket/item_features.parquet	features.retailrocket_item_features
retailrocket_recommendation_feature_registry	v1	retailrocket_item_features	v1	["item_id"]	data/features/source=retailrocket/item_features.parquet	features.retailrocket_item_features

10. Data Versioning and Lineage with DVC

DVC is used to track large datasets and model artifacts while Git stores source code, reports, and DVC metadata files. This provides reproducibility without committing heavy files directly to Git.

pipeline_layer	description	dvc_file	tracked_path	md5	size_bytes	nfiles	versioning_tool
external_data	Retailrocket original external CSV dataset	data/external/retailrocket.dvc	retailrocket	a9cdd04c0b079c0a7becea06b1f4c684.dir	578568116	3	DVC
raw_batch_data	Retailrocket raw batch ingestion snapshots	data/raw/source=retailrocket_batch.dvc	source=retailrocket_batch	102bada8dce214fb6e395264e2dbf5d9.dir	2892849190	30	DVC
raw_api_data	Retailrocket raw REST API catalog delta snapshots	data/raw/source=retailrocket_api.dvc	source=retailrocket_api	47eee925eea8fdadca1e0ea6d94dc05d.dir	146289	46	DVC
staged_data	Retailrocket staged Parquet data	data/staged/source=retailrocket.dvc	source=retailrocket	bbaa458b201f83802593b7561ab28d66.dir	79165589	7	DVC
curated_data	Retailrocket curated analytical Parquet data	data/curated/source=retailrocket.dvc	source=retailrocket	3465a819e7ed61c2c1058fa750e6d74e.dir	157299406	3	DVC
feature_data	Retailrocket feature Parquet data	data/features/source=retailrocket.dvc	source=retailrocket	0f135b7d2036a677afaeb07322fbefdf6.dir	144533617	3	DVC
model_artifact	Retailrocket trained recommendation model artifacts	models/retailrocket.dvc	retailrocket	f2720ee47bea2d8c546f4dc91d7dbfa3.dir	1226758	2	DVC

10.1 Repository Structure and Dataset Versioning

path	purpose	versioning	example_files
configs/feature_store/	Custom feature registry configuration	Git	retailrocket_feature_registry.json
data/external/	External source datasets	DVC metadata in Git	retailrocket.dvc
data/raw/	Raw Retailrocket ingestion snapshots	Ignored by Git; reproducible through ingestion	source=retailrocket_batch/, source=retailrocket_api/
data/staged/	Cleaned staged Parquet datasets	DVC metadata in Git	source=retailrocket.dvc
data/features/	ML feature datasets	DVC metadata in Git	source=retailrocket.dvc
data/warehouse/	Local DuckDB analytical warehouse	Generated artifact, ignored by Git	recomart.duckdb
data/curated/	Curated Retailrocket analytical datasets	DVC metadata when generated	source=retailrocket/
models/	Trained model artifacts	DVC metadata in Git	retailrocket.dvc
reports/	Reports, plots, screenshots, summaries	Git for lightweight files	final_project_report.md, plots/, screenshots/

path	purpose	versioning	example_files
sql/	Warehouse and feature transformation SQL	Git	retailrocket_warehouse.sql, retailrocket_features.sql
src/	Pipeline source code organized by stage	Git	ingestion/, validation/, preparation/, transformation/, training/, serving/
orchestration/	Prefect pipeline definitions	Git	retailrocket_pipeline.py, retailrocket_api_ingestion_flow.py

11. Model Training and Evaluation

The Retailrocket modeling stage includes two recommenders: an event-weighted global popularity baseline and a category/content-based filtering model. Event weights are: view = 1, addtocart = 3, transaction = 5. A time-based train/test split is used.

model_name	model_type	top_k	test_window_days	evaluation_sample_users	min_time_stamp_ms	max_time_stamp_ms	cutoff_time_stamp_ms	min_event_datetime	max_event_datetime	train_event_count	test_event_count
retailrocket_popularity_recommender	event_weighted_global_popularity	10	14	10000	1430622004384	1442545187788	1441335587788	2015-05-03 08:30:04.384000+05:30	2015-09-18 08:29:47.788000+05:30	2509138	246963

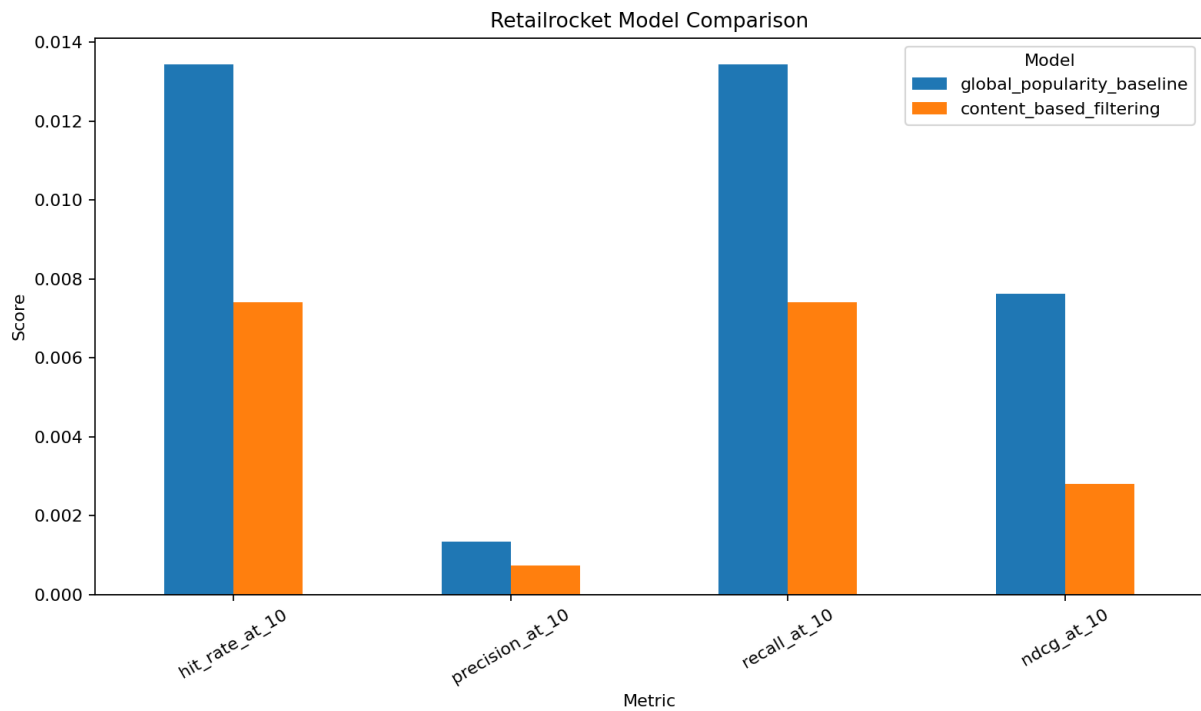
11.2 Content-Based Recommender

A second Retailrocket model was implemented using category/content-based filtering. It builds user profiles from historical item categories and parent categories, then scores candidate items using category similarity, availability metadata, conversion signals, and a popularity prior. This directly addresses the assignment requirement for content-based filtering.

model_name	model_type	top_k	test_window_days	evaluation_sample_users	min_time_stamp_ms	max_time_stamp_ms	cutoff_time_stamp_ms	candidate_item_count	test_target_users	user_profile_count	evaluated_users
retailrocket_content_based_recommender	category_content_based_filtering	10	14	10000	1430622004384	1442545187788	1441335587788	5000	3648	517	3648

11.3 Model Comparison

model_family	model_name	model_type	top_k	evaluated_users	hits	hit_rate_at_10	precision_at_10	recall_at_10	ndcg_at_10
global_popularity_baseline	retailrocket_popularity_recommender	event_weighted_global_popularity	10	3648	49	0.0134320175438596	0.0013432017543859	0.0134320175438596	0.0076286552952891
content_based_filtering	retailrocket_content_based_recommender	category_content_based_filtering	10	3648	27	0.0074013157894736	0.0007401315789473	0.0074013157894736	0.0028152506650017



Comparison of popularity baseline and content-based recommender metrics.

Retailrocket performance: HitRate@10 = 0.013432, Precision@10 = 0.001343, Recall@10 = 0.013432, NDCG@10 = 0.007629.

NDCG@10 is ranking-aware: a relevant item at rank 1 receives more credit than one at rank 10.

12. MLflow Tracking Evidence

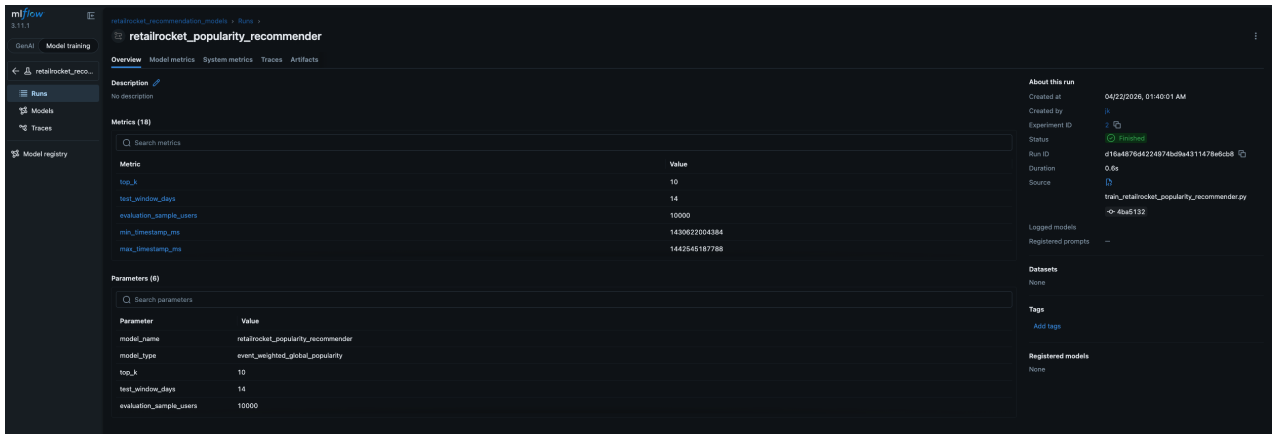
MLflow stores run IDs, parameters, metrics, and artifacts. Screenshots below show successful experiment tracking for Retailrocket models.

Name	Time created	Last modified	Description	Tags
retailrocket_recommendation_models	04/21/2026, 11:37:06 PM	04/21/2026, 11:37:09 PM	-	-
dummy_recommender_baselines	04/21/2026, 02:52:49 AM	04/21/2026, 02:52:49 AM	-	-
Default	04/21/2026, 02:52:49 AM	04/21/2026, 02:52:49 AM	-	-

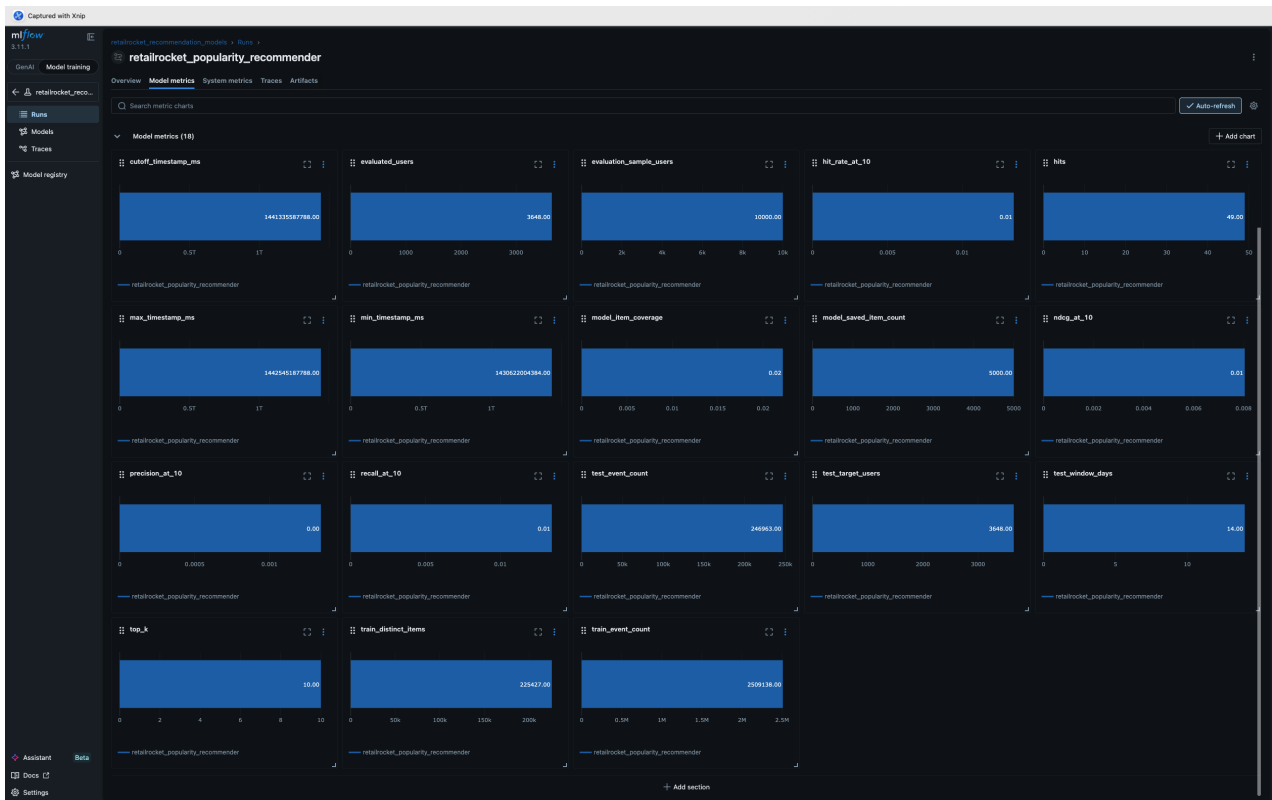
MLflow experiments list.

Run Name	Created	Dataset	Duration	Source	Models	hit_rate_at_10	precision_at_10	recall_at_10	ndcg_at_10	test_event_count	test_target_users	train_event_count
retailrocket_popularity_recommender	3 minutes ago	-	0.6s	train_retailrocket_popularity_recommender.py	-	0.01343201754385965	0.00046062757233324	0.001343201754385965	0.01343201754385965	246963	3648	2509158
retailrocket_popularity_recommender	1 hour ago	-	67ms	train_retailrocket_popularity_recommender.py	-	0.01343201754385965	0.001343201754385965	0.01343201754385965	0.01343201754385965	246963	3648	2509158
retailrocket_popularity_recommender	1 hour ago	-	71ms	train_retailrocket_popularity_recommender.py	-	0.01343201754385965	0.001343201754385965	0.01343201754385965	0.01343201754385965	246963	3648	2509158
retailrocket_popularity_recommender	1 hour ago	-	79ms	train_retailrocket_popularity_recommender.py	-	0.01343201754385965	0.001343201754385965	0.01343201754385965	0.01343201754385965	246963	3648	2509158
retailrocket_popularity_recommender	2 hours ago	-	65ms	train_retailrocket_popularity_recommender.py	-	0.01343201754385965	0.001343201754385965	0.01343201754385965	0.01343201754385965	246963	3648	2509158

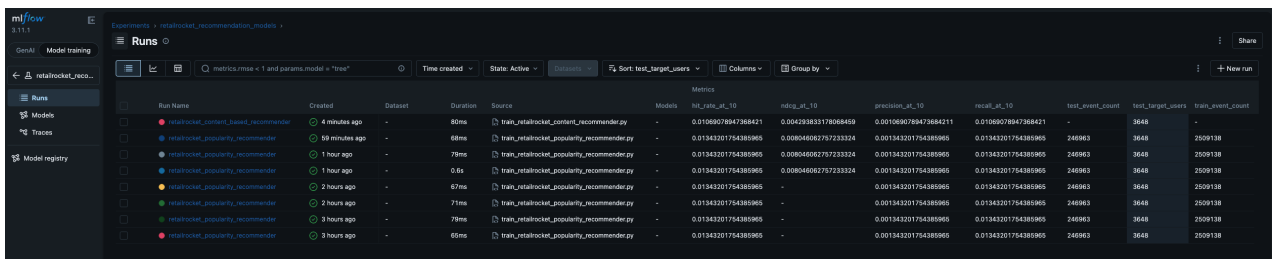
Retailrocket runs table with metrics.



Retailrocket run overview.



Retailrocket metric details.



Retailrocket content-based recommender run in MLflow.

13. Inference Demo

The inference script loads the model and generates recommendations for default active users, one user, or custom comma-separated users. Previously seen items are filtered.

model_name	model_type	requested_user_count	top_k	recommendation_rows	distinct_recommended_items	training_hit_rate_at_10	training_precision_at_10	training_recall_at_10	test_target_users
retailrocket_popularity_recommender	event_weighted_global_popularity	5	5	25	18	0.0134320175438596	0.0013432017543859	0.0134320175438596	3648

model_name	model_type	requested_user_count	top_k	recommendation_rows	distinct_recommended_items	training_hit_rate_at_10	training_precision_at_10	training_recall_at_10	test_target_users
retailrocket_content_based_recommender	category_content_based_filtering	5	5	25	7	0.0074013157894736	0.0007401315789473	0.0074013157894736	3648

14. Pipeline Orchestration

Prefect orchestrates the Retailrocket-only pipeline. The flow runs batch ingestion, REST or mock API delta ingestion, raw validation, staged and curated preparation, warehouse loading, feature building, feature retrieval, model training, inference, comparison, and report generation.

step_order	step_name	status	duration_seconds
1	inspect_retailrocket_external	PASS	6.162
2	ingest_retailrocket_batch	PASS	23.525
3	ingest_retailrocket_catalog_api_delta	PASS	2.691
4	validate_retailrocket_raw	PASS	13.203
5	prepare_retailrocket_staged_and_curated	PASS	8.927
6	validate_retailrocket_staged_and_curated	PASS	1.896
7	load_retailrocket_to_duckdb	PASS	14.638
8	build_retailrocket_features	PASS	11.76
9	retrieve_retailrocket_features	PASS	13.266
10	train_retailrocket_popularity_recommender	PASS	58.73
11	train_retailrocket_content_recommender	PASS	99.372
12	run_retailrocket_popularity_inference	PASS	0.827
13	run_retailrocket_content_based_inference	PASS	0.885
14	generate_model_comparison	PASS	1.696
15	generate_assignment_evidence	PASS	2.043
16	generate_final_report	PASS	0.619
17	generate_assignment_pdf	PASS	2.341

15. Reproducibility Workflow

- Activate environment: `conda activate recomart`
- Check version state: `git status` and `dvc status`
- Restore tracked artifacts if remote is configured: `dvc pull`
- Run Retailrocket flow: `python -m orchestration.retailrocket_pipeline`
- Run real API ingestion: `python -m src.ingestion.ingest_retailrocket_catalog_api`
- Run explicit real API ingestion: `RECOMART_CATALOG_API MOCK_MODE=false RECOMART_CATALOG_API_BASE_URL=https://recomart-flask.295uyonmxxer.us-south.codeengine.appdomain.cloud RECOMART_CATALOG_API_ENDPOINT=/items RECOMART_CATALOG_API_PAGE_SIZE=50 python -m src.ingestion.ingest_retailrocket_catalog_api`
- Run mock fallback: `RECOMART_CATALOG_API MOCK_MODE=true python -m src.ingestion.ingest_retailrocket_catalog_api`
- Schedule REST ingestion every 30 minutes: `prefect deploy orchestration/retailrocket_api_ingestion_flow.py:retailrocket_api_ingestion_flow --name retailrocket-api-every-30-minutes --interval 1800 --pool default-agent-pool`

- Open MLflow UI using the tracking URI from Python: `mlflow ui --backend-store-uri "$TRACKING_URI" --port 5001`

16. Operational Notes

- Current Retailrocket models are lightweight and suitable for the assignment scope.
- The content-based model is the main assignment model; popularity is retained as a benchmark.
- The API ingestion flow uses the teammate API by default and supports local mock fallback by environment variable.
- The real API serves new rows without repetition; the ingestion client records API count, total rows, unread rows, and success metadata.
- Generated heavy data and model files are DVC-tracked rather than committed directly to Git.
- The Prefect API ingestion deployment command documents the 30-minute periodic metadata refresh.

17. Conclusion

The project implements a complete and reproducible ML data management pipeline for recommendation systems using Python, DuckDB, Parquet, DVC, MLflow, and Prefect. It satisfies the required stages from ingestion through orchestration and produces source code, reports, screenshots, and a final PDF.